

Análise de Crimes em Chicago

Pipeline Big Data com Apache Spark

7,9M

registros

1,9 GB

dados brutos

23

colunas

3

modelos ML

Apache Spark 3.5

PySpark MLlib

Docker Compose

Parquet

Classificação binária — prever se um crime resultará em prisão

Objetivo e Escolha do Problema

O que queremos prever?

Dado um registro de crime em Chicago, predizer se haverá **prisão** ou não — variável **Arrest**: 0 (sem prisão) ou 1 (com prisão).

É um problema de **classificação binária supervisionada** com dados reais de segurança pública de 2001 a 2017.

Por que esse dataset?

- Cobre **16 anos** de registros policiais reais de Chicago
- **1,9 GB** de dados — atende o requisito de Big Data
- Rica em features: tipo de crime, local, horário, distrito, coordenadas
- Desbalanceamento de classes realista — exige tratamento explícito
- Disponível publicamente no **Kaggle**

A pergunta "esse crime vai resultar em prisão?" tem valor prático: entender quais fatores mais influenciam o resultado ajuda a identificar padrões de policiamento e efetividade por tipo de crime e região.

Parte 1 — Ambiente de Big Data com Docker

Arquitetura do cluster

Container	Função	Recursos
spark-master	Coordena o cluster	—
spark-worker-1	Processa partições	2 cores · 2 GB
spark-worker-2	Processa partições	2 cores · 2 GB
jupyter	Driver PySpark	2 GB

Monitoramento em tempo real via Spark UI em `localhost:4040`

Por que Docker e por que Spark?

- **Docker** garante reprodutibilidade total — mesmo ambiente em qualquer máquina, sem conflito de dependências
- **Docker Compose** orquestra os 4 containers com um único comando

- **Apache Spark** processa dados em memória de forma distribuída — escala com o volume sem reescrever o código
- Os 2 workers processam partições do dataset **em paralelo**, reduzindo o tempo de execução

O cluster simula um ambiente de produção real — o mesmo código rodaria em AWS EMR ou Databricks sem modificações.

O Dataset — Chicago Crimes (Kaggle)

4 arquivos CSV — total 1,9 GB

Período	Tamanho
2001–2004	453 MB
2005–2007	449 MB
2008–2011	646 MB
2012–2017	350 MB

O que cada linha representa

Um registro policial: tipo do crime, data e hora, endereço, distrito, se houve prisão, coordenadas GPS e descrição do local.

Colunas utilizadas no modelo

Coluna	Tipo	Papel
Primary Type	Categórica	Feature
Location Description	Categórica	Feature
District / Beat	Numérica	Feature

Latitude / Longitude	Numérica	Feature
Domestic	Booleana	Feature
Hour / DayOfWeek / Month	Numérica	Feature (derivada)
Arrest	Booleana	Target (0/1)

Parte 1 — Ingestão e Conversão para Parquet

4 CSVs (1,9 GB)



Leitura paralela no Spark



DataFrame unificado



Gravação Parquet



551 MB (comprimido)

Por que converter para Parquet?

- **Formato colunar** — lê somente as colunas solicitadas, ignorando o restante. CSV lê tudo, sempre
- **Compressão Snappy integrada** — 1,9 GB → 551 MB sem perda de dados (70% menor)
- **Schema embutido** — os tipos já estão salvos, sem necessidade de re-inferência

- **Leitura 5–10× mais rápida** nas etapas de ETL e ML seguintes

Por que isso importa neste projeto?

O pipeline tem **4 fases** que leem os dados.

Converter para Parquet uma vez e reler nas fases seguintes economizou dezenas de minutos de I/O. Com CSV, cada fase releria 1,9 GB do disco. Com Parquet, lê apenas as colunas necessárias em 551 MB já comprimidos.

Decisão de engenharia padrão em pipelines Big Data de produção.

Parte 2 — Pré-processamento: Limpeza dos Dados

Etapa	O que foi feito	Por que foi necessário	Registros restantes
Leitura do Parquet	7.941.286 registros carregados	Ponto de partida	7.941.286
Remoção de nulos	dropna() nas colunas críticas	Linhas sem tipo de crime, data ou resultado de prisão são inutilizáveis para qualquer análise ou modelo	7.834.133
Filtro de coordenadas	Remove lat/lon = 0	Coordenada (0,0) é o oceano Atlântico — indica ausência de dado geográfico, não localização real	—

Seleção de colunas	Mantém 10 colunas relevantes	ID, Case Number, IUCR e FBI Code não têm poder preditivo e aumentam custo de processamento	—
Dropna final	Remove nulos restantes nas features	O modelo de ML exige que todas as features de entrada estejam presentes	7.145.306

796.000 registros removidos (~10%) — principalmente por ausência de coordenadas geográficas válidas. Essa perda é aceitável: os registros descartados não carregam informação utilizável.

Parte 2 — Feature Engineering

Features derivadas do timestamp

Feature criada	De onde vem	Por que importa
Hour	Hora da ocorrência	Prisões têm padrão circadiano claro — pico às 19–21h (~22%)
DayOfWeek	Dia da semana	Fins de semana têm perfil de crimes diferente de dias úteis
Month	Mês do ano	Captura sazonalidade — alguns crimes concentram em épocas específicas

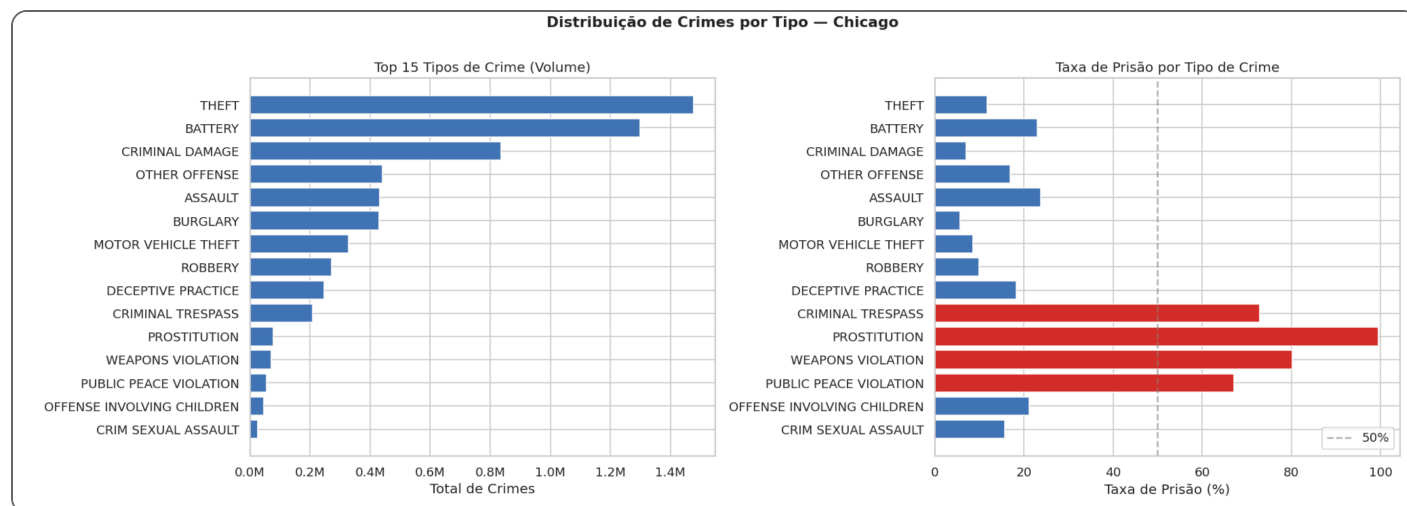
O timestamp bruto não pode entrar no modelo — é texto. Extrair componentes numéricos transforma informação temporal em features utilizáveis.

Outras transformações

- **Arrest_label** — converte "True"/"False" (string) para 1/0 (inteiro). O modelo exige numérico
- **Domestic_int** — mesma lógica para a coluna de violência doméstica
- **District, Beat, Community Area** — convertidos de string para inteiro
- **Latitude e Longitude** — convertidos de string para double para entrar no VectorAssembler

Essas conversões são obrigatórias — o Spark MLlib não aceita tipos mistos no vetor de features.

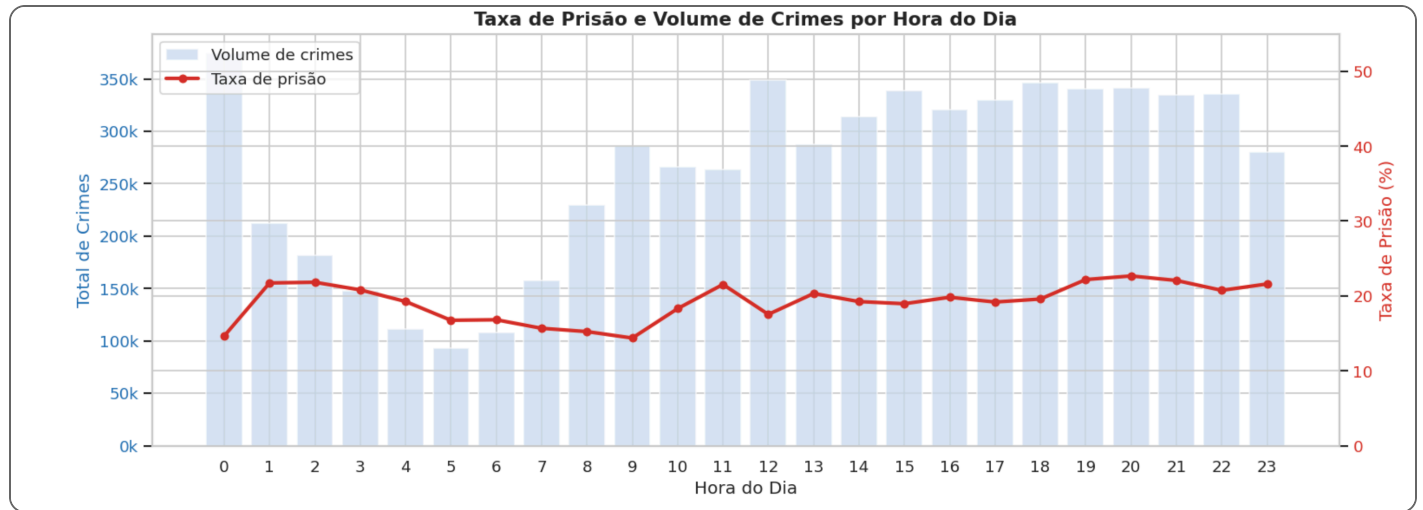
EDA — Distribuição por Tipo de Crime



Theft e Battery lideram em volume — juntos representam mais de 35% de todos os crimes, mas com taxa de prisão baixa (12% e 23%), pois geralmente ocorrem sem testemunhas imediatas.

PROSTITUTION (100%) e WEAPONS VIOLATION (82%) — maiores taxas de prisão após remoção do NARCOTICS. CRIMINAL TRESPASS (73%) lidera a feature importance exatamente por isso.

EDA — Taxa de Prisão por Hora do Dia

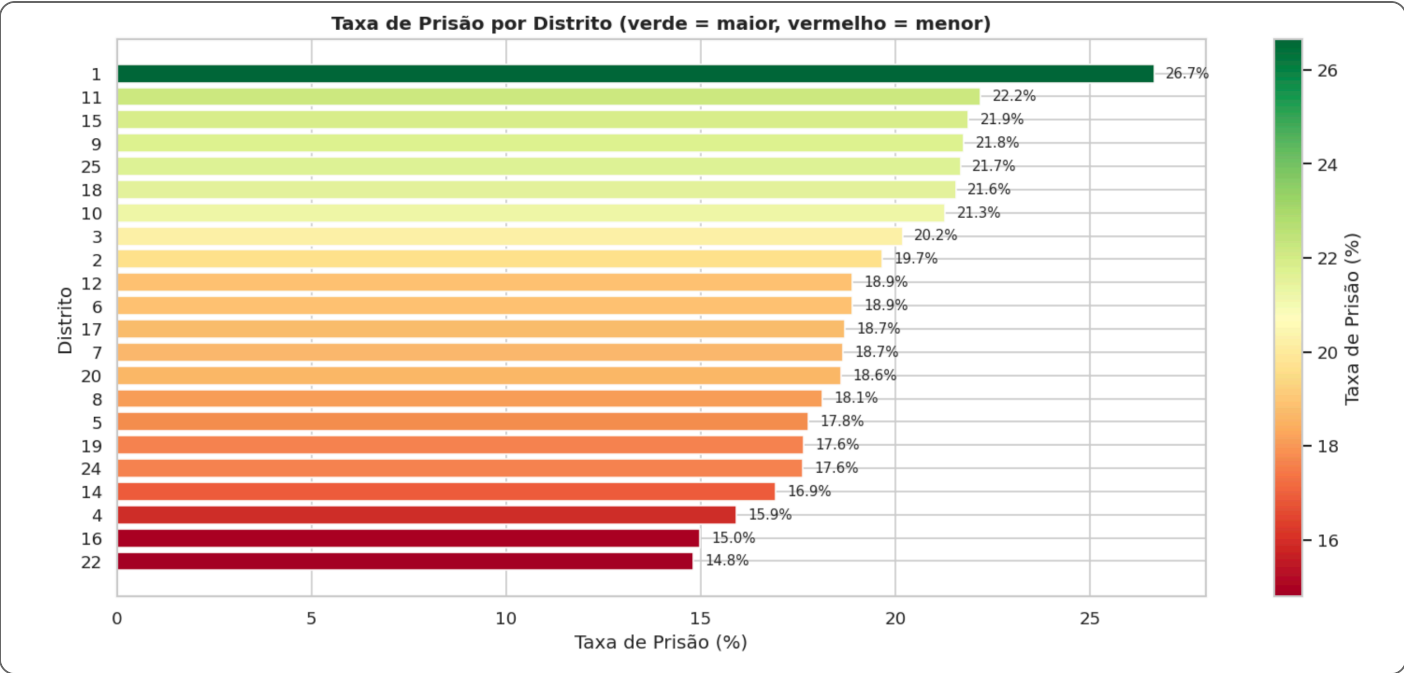


Barras azuis: volume total de crimes · Linha vermelha: taxa de prisão (%)

Pico às 20h (~22,7%) — não é o horário de maior volume de crimes. Indica ação policial ativa no período noturno.

Menor taxa às 9h (~14,4%) — horário comercial com alto volume de ocorrências mas menor proporção de flagrantes.

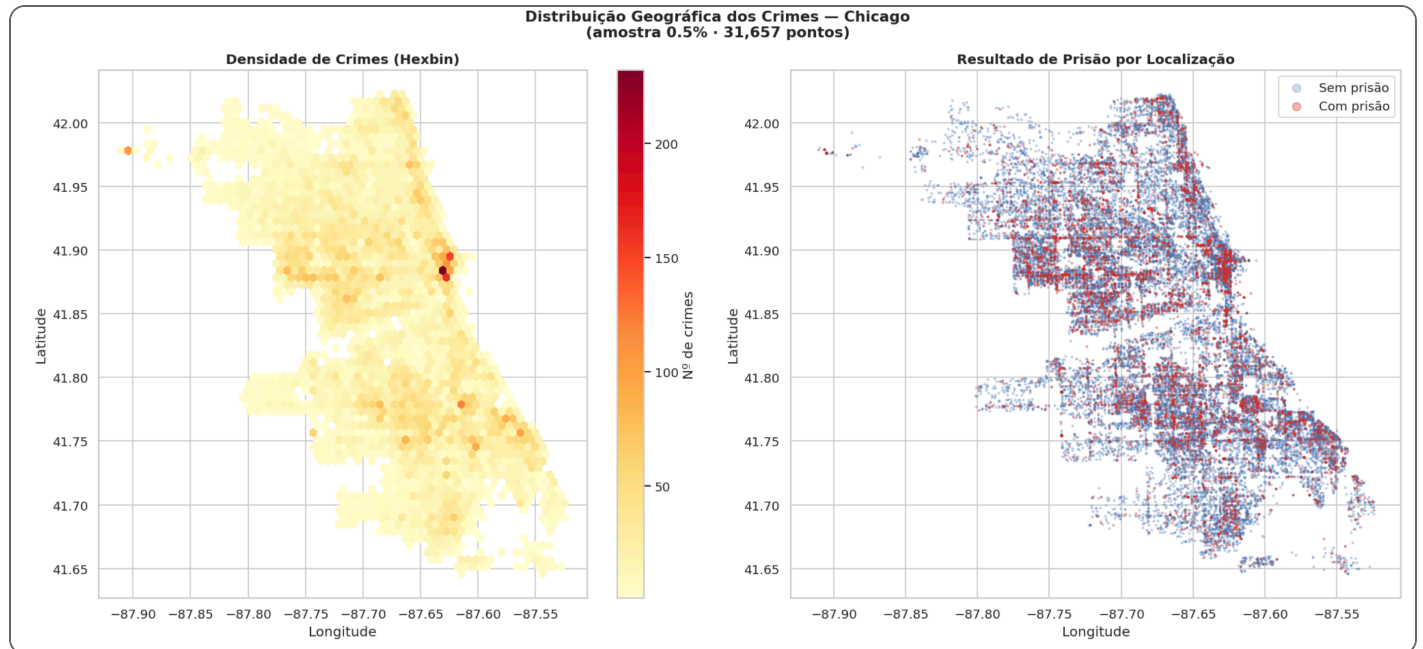
EDA — Taxa de Prisão por Distrito



Distrito 1 (26,7%) e Distrito 11 (22,2%) — maior efetividade de prisão. Concentram ocorrências com alta probabilidade de detenção no centro e região norte.

Distrito 22 (14,8%) e Distrito 16 (15,0%) — menor taxa. Variação de 1,8x entre distritos revela desigualdade de efetividade policial e perfil criminal distinto.

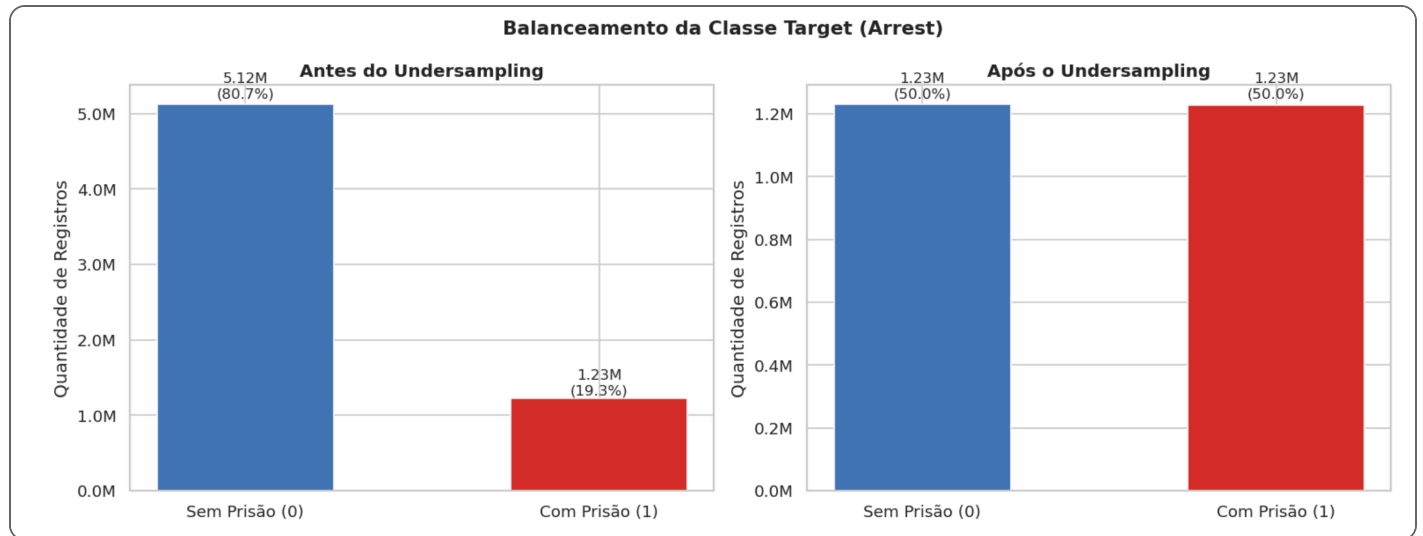
EDA — Distribuição Geográfica dos Crimes



Amostra de 0,5% dos registros (~31.657 pontos) · Azul: sem prisão · Vermelho: com prisão

A concentração de crimes é maior no centro-sul de Chicago. Crimes com e sem prisão seguem a mesma distribuição espacial — **não há separação geográfica clara entre as classes**. A localização sozinha não é um bom preditor, mas contribui em combinação com outras features.

Pré-processamento — Balanceamento de Classes



Por que o desbalanceamento é um problema?

Com 71,8% de casos "sem prisão", um modelo que *sempre prevê zero* acertaria 71,8% sem aprender absolutamente nada. As métricas ficariam boas sem nenhuma inteligência real.

Solução: Undersampling

Reduzimos a classe majoritária (5,1M) para igualar a minoritária (2,0M), resultando em **4,0M de registros balanceados (50/50%)**.

Preferimos undersampling a dados sintéticos (SMOTE) — mantemos apenas registros reais, sem risco de introduzir ruído artificial.

Por que Undersampling e não SMOTE?

SMOTE — Synthetic Minority Oversampling

Cria registros artificiais na classe minoritária interpolando entre amostras reais. É uma abordagem válida, mas aqui temos dois problemas concretos que nos fizeram descartá-la.

- A classe minoritária já tem **2 milhões de registros reais** — não há escassez de exemplos para justificar dados sintéticos
- Com 203 dimensões de features OHE esparsas, a interpolação entre dois registros pode gerar combinações impossíveis na prática (ex.: crime às 14h em dois distritos ao mesmo tempo)
- O Spark MLlib não tem SMOTE nativo — a implementação manual sobre 7M linhas seria inviável no tempo do projeto

Undersampling — nossa escolha

Reduzimos a classe majoritária de 5,1M para 2M por amostragem aleatória, totalizando 4M registros balanceados.

- Todos os registros são **dados reais do dataset** — sem ruído introduzido artificialmente
- Mesmo descartando ~3,9M linhas, ainda temos volume suficiente para treino (1,97M no split final)
- Os padrões relevantes da classe majoritária continuam representados — a amostragem é aleatória com seed fixo para reprodutibilidade

Conclusão: SMOTE compensa quando a classe minoritária tem centenas ou poucos milhares de exemplos. Com 2 milhões de casos reais de prisão disponíveis, undersampling é a escolha mais limpa.

Parte 3 — Pipeline de Machine Learning



StringIndexer

Converte categorias de texto em índices numéricos ordenados por frequência.

Necessário porque algoritmos de ML trabalham apenas com números — não entendem texto diretamente.

OneHotEncoder

Transforma cada índice em vetor binário esparsos, eliminando a falsa ordem numérica.

Sem isso, o modelo entenderia "THEFT=0 é menor que BATTERY=1" — o que não faz sentido.

VectorAssembler

Concatena todas as features em um único vetor de **203 dimensões**.

Split: 80% treino (3,23M linhas) · 20% teste (807K linhas) · seed fixo para reprodutibilidade.

O pipeline é treinado (*fit*) apenas no conjunto de treino e aplicado (*transform*) no teste — sem data leakage.

Os 3 Modelos Escolhidos

Logistic Regression

Baseline

Modelo linear clássico. Estabelece a referência mínima — qualquer modelo mais complexo deve superar esse resultado.

Interpretável e escalável: o Spark distribui o gradiente entre os workers sem carregar tudo na memória de uma vez.

Decision Tree

Interpretável

Captura relações não-lineares sem necessidade de normalização. A principal vantagem é a **feature importance** — revela quais variáveis mais influenciam a decisão.

Profundidade máxima 5 para evitar overfitting.

MLP Neural Net

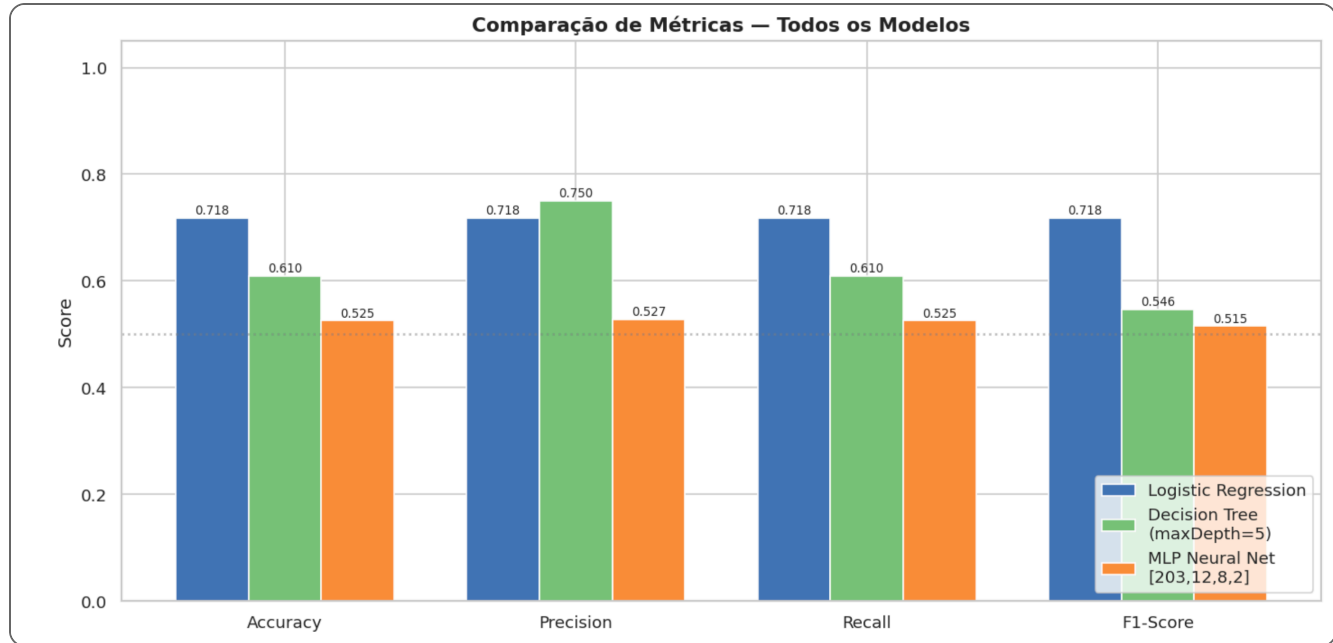
Rede Neural

Arquitetura com 4 camadas: $203 \rightarrow 12 \rightarrow 8 \rightarrow 2$.

Capaz de aprender padrões não-lineares complexos.

Limitação: o otimizador L-BFGS carrega o dataset inteiro por iteração — inviável com 3,2M linhas em 4 GB de RAM.

Resultados — Comparação dos Modelos



Logistic Regression

71,8%

Accuracy · F1: 0,718

Melhor modelo geral — equilibrado em todos os erros

Decision Tree

61,0%

Accuracy · F1: 0,546

Melhor Precision (75,0%) — muito poucos falsos positivos

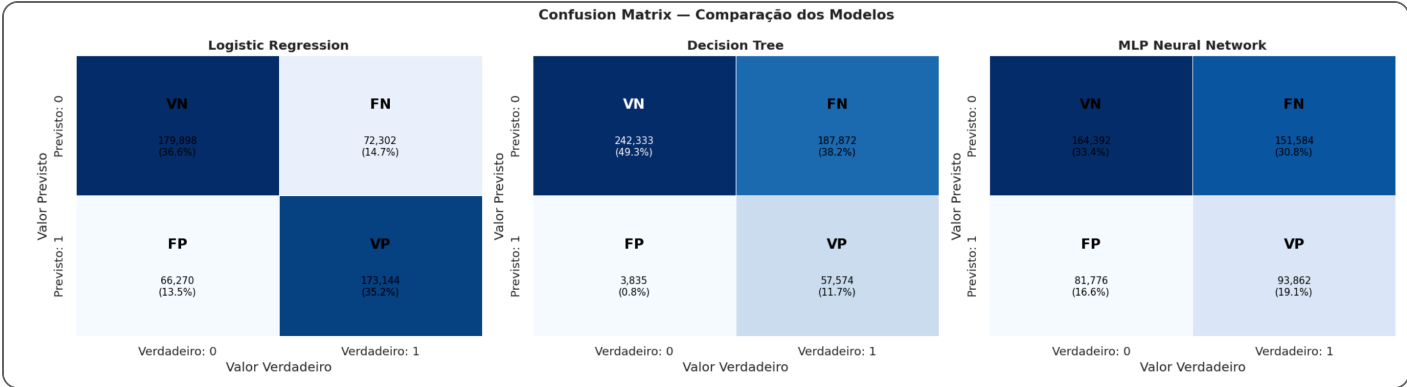
MLP Neural Net

52,5%

Accuracy · F1: 0,515

Próximo do acaso (50%) — treinado em apenas 10% dos dados

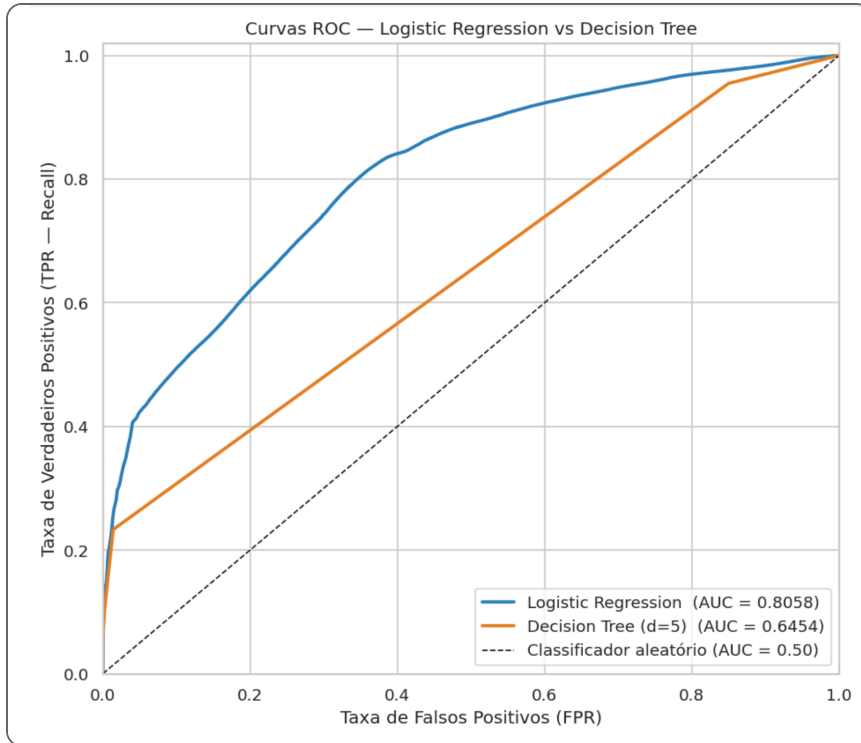
Análise de Erros — Confusion Matrix



LR vs DT — estratégias opostas: a LR equilibra os erros (FP: 13,5% · FN: 14,7%) e mantém 35,2% de acertos reais de prisão. A DT é extremamente conservadora — apenas 0,8% de FP, mas erra 38,2% das prisões reais. Sem o atalho do NARCOTICS, a DT com depth 5 simplesmente chuta "sem prisão" na maioria dos casos.

MLP — distribuição próxima do aleatório: com 10% dos dados e 20 iterações, o modelo converge para um limiar que erra proporcionalmente nos dois lados (FP: 16,6% · FN: 30,8%). Não é completamente aleatório, mas não aprendeu padrões úteis.

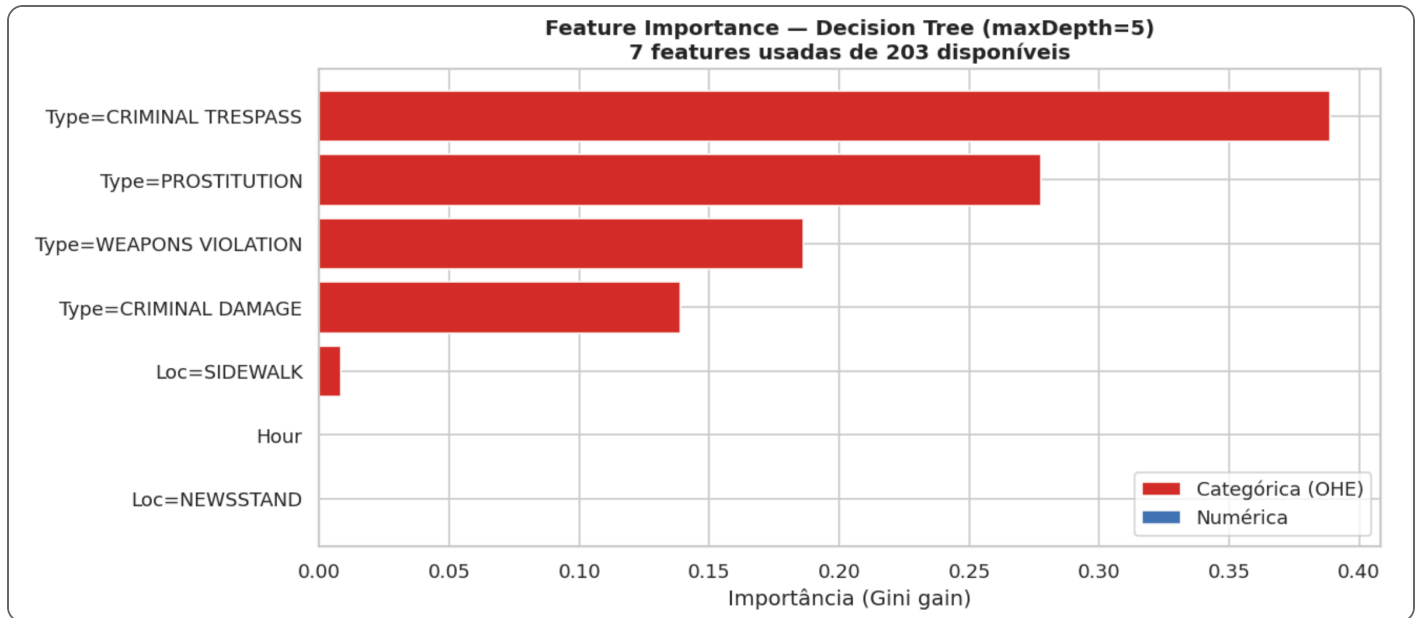
Curvas ROC — Discriminação dos Modelos



Logistic Regression — AUC = 0,8058: discrimina bem entre classes; mantém alta TPR com baixo FPR no trecho inicial da curva.

Decision Tree — AUC = 0,6454: com apenas 31 folhas, o modelo tem poucos limiares de decisão — curva linear, próxima do aleatório.

O que o Modelo Aprendeu — Feature Importance



CRIMINAL TRESPASS lidera com 0,39 de Gini gain, seguido por PROSTITUTION (0,27) e WEAPONS VIOLATION (0,19) — todos com taxas de prisão reais acima de 70%. Sem o atalho do NARCOTICS, a árvore aprendeu a distinguir crimes por categorias com padrões genuínos de enforcement.

7 de 203 features foram usadas — com depth 5 a árvore ainda ignora features numéricas (Hour, District). Isso explica o 38,2% de falsos negativos: ela sabe quais tipos de crime costumam resultar em prisão, mas não refina pela hora, local ou distrito.

Análise Crítica dos Resultados

Por que a Logistic Regression superou a Decision Tree?

- A Decision Tree com profundidade 5 cria apenas 31 folhas — com 203 features, isso é muito restritivo
- A árvore usou apenas 7 features; a LR considera todas as 203
- Um modelo simples venceu um complexo porque o complexo estava mal configurado
- Com profundidade 10–15 a DT provavelmente inverteria o resultado

Por que o MLP ficou em 52,5%?

- Treinado em apenas **10% dos dados** por limitação de memória
- O otimizador L-BFGS carrega o dataset inteiro por iteração

- Redes neurais precisam de muito mais dados e épocas para convergir

O que mudou ao remover NARCOTICS

NARCOTICS foi removido do pipeline por ter 99,21% de taxa de prisão — um atalho que distorcia feature importance e métricas gerais.

- LR caiu de 79,9% → 71,8%: a queda mostra o quanto o NARCOTICS inflava as métricas anteriores
- DT caiu de 76,8% → 61,0% e ganhou 38,2% de falsos negativos: sem o split fácil no primeiro nível, depth 5 não tem capacidade para capturar os padrões restantes
- Feature importance agora reflete crimes reais: CRIMINAL TRESPASS, PROSTITUTION e WEAPONS VIOLATION

Os modelos atuais são mais honestos: o que aprenderam se generaliza para crimes reais, não para um caso trivial de 99%.

Desafios Enfrentados

SparkContext morreu por falta de memória

Durante o treino do MLP, os 4 GB de RAM do cluster se esgotaram e o SparkContext foi encerrado silenciosamente. Execuções seguintes falhavam com *AssertionError: sc is not None*.

Solução: recovery guard — o código verifica se o Spark está vivo antes do MLP e reinicia automaticamente, recarregando os dados do checkpoint em Parquet.

MLP travava indefinidamente

Com 3,2M linhas e 203 features, o L-BFGS tentava carregar o dataset inteiro por iteração. A célula ficava com *[*]* sem progredir por horas.

Solução: amostrar 10% do treino (~323K linhas) e reduzir de 50 para 20 iterações.

Encoding inconsistente quebrou o MLP

O pipeline treinado na amostra de 10% aprendeu 170 categorias ao invés de 203. Ao transformar o teste completo: *ArrayIndexOutOfBoundsException: index 203 out of bounds for double[170]*.

Solução: separar encoding do treino. O pipeline de features é ajustado no dataset completo (203 features). Apenas o classificador MLP treina na amostra.

Tempo de execução elevado

Cada execução completa levou **50 a 70 minutos**. Foram necessárias várias execuções para depurar e validar o pipeline.

Estratégia: checkpoint do dataset balanceado em Parquet. Em caso de falha, reiniciamos apenas a partir da Parte 3 — sem reprocessar os 7,9M de registros.

Limitações Honestas

Do dataset

- **Dados históricos (2001-2017)** — padrões de policiamento e legislação mudaram
- **Crimes não reportados são invisíveis** — o dataset representa o que foi registrado, não o que aconteceu de fato. Viés de seleção inevitável
- **107.000 registros sem coordenadas válidas** foram descartados
- **NARCOTICS removido do pipeline** — 99% de taxa de prisão criava um atalho trivial que distorcia métricas e feature importance; os modelos foram retreinados sem essa categoria
- **Sem dados socioeconômicos** por bairro — renda e densidade teriam poder preditivo real

Dos modelos e do hardware

- **MLP com 10% dos dados** — o resultado de 53% não é comparável com LR e DT em condições iguais

- **Decision Tree com profundidade 5** — subutiliza as 203 features disponíveis
- **Split aleatório em vez de cronológico** — o correto para séries temporais seria treinar em 2001–2015 e testar em 2016–2017; o impacto aqui é limitado porque padrões de crime por tipo e distrito são estáveis ao longo dos 16 anos, mas é uma inconsistência metodológica real
- **Sem cross-validation (k-fold)** — avaliação com split único tem variância alta
- **Sem tuning de hiperparâmetros** — nenhum GridSearch por custo computacional
- **Cluster local com 4 GB de RAM total** — impede treino do MLP completo e uso de modelos ensemble

O que Gostaríamos de ter Feito

Modelos não implementados

- **Random Forest e Gradient Boosted Trees** — modelos ensemble que quase sempre superam uma árvore isolada. Descartados por uso intensivo de memória
- **MLP no dataset completo** — com cluster maior ou mini-batch SGD, treinaria nos 3,2M registros
- **Cross-validation com k=5** — estimativas muito mais confiáveis, mas multiplica o tempo por 5

Análises não realizadas

- **Métricas separadas por tipo de crime** — analisar performance isolada por categoria
- **Cross-validation k-fold** — estimativas com variância menor que um único split
- **Tuning de hiperparâmetros (GridSearch)** — explorar profundidade da DT e regularização da LR

Por que não fizemos?

Cada decisão foi **consciente**, não por falta de conhecimento:

- **Hardware:** 4 GB de RAM total é o principal limitante. Random Forest esgotaria a memória
- **Tempo:** cada execução leva 20–40 minutos. Com múltiplas execuções para depuração, não havia margem para experimentos adicionais
- **Escopo:** o foco foi construir um pipeline completo e funcional de ponta a ponta

Em AWS EMR ou Databricks, todos esses pontos seriam viáveis com o mesmo código — apenas com mais recursos de hardware.

Conclusões

Resultados finais

Modelo	Accuracy	Precision	F1-Score
Logistic Regression	71,8%	71,8%	0,718
Decision Tree (d=5)	61,0%	75,0%	0,546
MLP [203,12,8,2]	52,5%	52,7%	0,515

Treino: 1,97M linhas · Teste: 492K linhas · 203 features · sem NARCOTICS

Pipeline funcional de ponta a ponta: 1,9 GB de CSV → ETL → EDA → 3 modelos treinados e avaliados com 7,9M de registros em cluster Spark distribuído.

O que aprendemos

- **Formato importa:** Parquet reduziu 1,9 GB para 551 MB e acelerou leituras em 5–10×
- **Balanceamento é não-negociável:** sem undersampling, qualquer modelo aprenderia a dizer "sem prisão" e acertaria 80,7% sem inteligência alguma

- **Hardware limita algoritmo:** a escolha do modelo foi parcialmente determinada pela RAM, não só pelo desempenho teórico
- **Modelos simples bem alimentados vencem redes neurais mal alimentadas:** LR com 203 features superou MLP treinado em 10% dos dados
- **Feature importance é tão valiosa quanto accuracy:** revelou que sem NARCOTICS a árvore distribui o peso entre tipos de crime com padrões reais de enforcement

Obrigado

7,9M

linhas processadas

1,9 GB

dados brutos

71,8%

melhor accuracy

203

features no vetor ML

Apache Spark 3.5

PySpark MLlib

Docker Compose

Parquet + Snappy

Logistic Regression

Decision Tree

MLP Neural Net

Speaker notes